Real-Time Operating System (RTOS)

Prof. Kurian Polachan, IIIT-Bangalore

The Need for RTOS

Task Scheduling

- Toggle an LED every 1 second.
- Toggle another LED every 2 second.
- Toggle another LED every 1.5 second.
- Toggle couple of other LED's 3 seconds, 4.1 seconds, and 4.5 seconds
- Add breathing effect to an LED. (i.e., increase/decrease the intensity of LED over time)
- Add …



RTOS Basics

Prof. Kurian Polachan, IIIT-Bangalore

RTOS – How To ?

Define Tasks → Create Tasks → Call the Scheduler

```
task1()
{
    loop:
        toggle LED1
        dealy(1s)
}

task2()
{
    loop:
        Toggle LED2
        Delay(2s)
}

task3()
{
    loop:
        Toggle LED3
        Delay(3.2s)
}
```

```
create(task1)
create(task2)
create(task3)

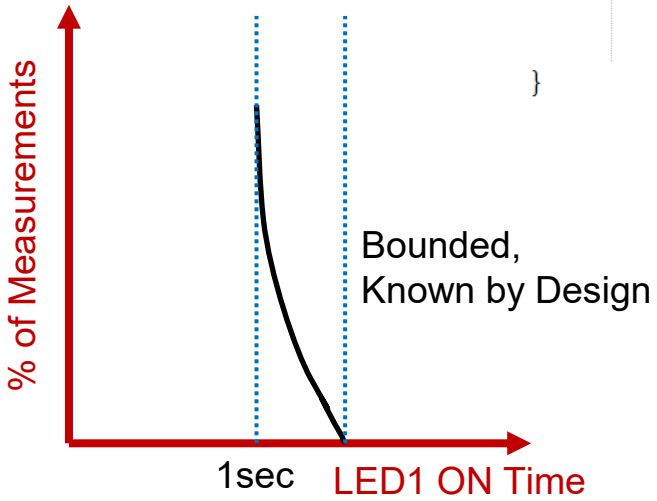
start scheduler
```

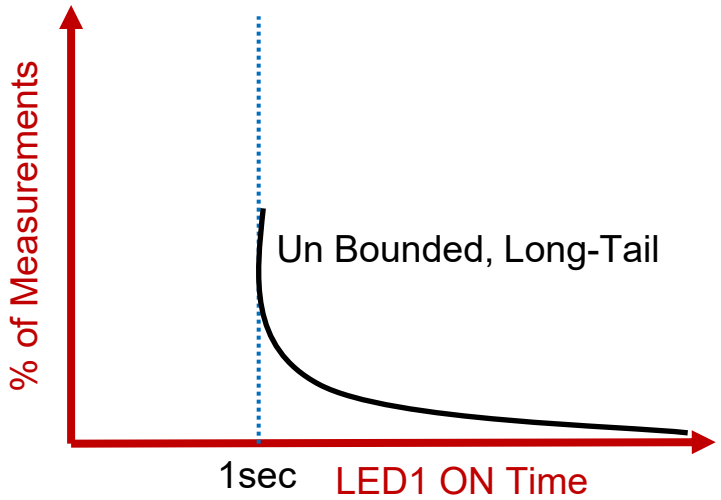
task-1, task-2, and task-3 toggles LED-1,2, and 3 every 1, 2 and 3.2 seconds, respectively.

Question: RTOS vs OS. What's the difference ?

RTOS vs. Generic OS

```
task1()
{
    loop:
        turn ON LED
        delay(1s)
        turn OFF LED
        delay(1s)
}
```





FreeRTOS Code

Define Tasks → Create Tasks → Call the Scheduler

```c
void led_task()
{
    while(1)
    {
        uint8 led_state = !led_Read();
        led_Write(led_state);
        vTaskDelay(250);
    }
}

void uart_task()
{
    uint16 counter = 1;
    char msg[32];

    while (1)
    {
        sprintf(msg, "Live %d\r\n", counter);
        UART_PutString(msg);

        counter++;

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

```c
int main(void)
{
    CyGlobalIntEnable;

    UART_Start();
    UART_ClearTxBuffer();

    FreeRTOS_Start();

    xTaskCreate(led_task, "LED_Task", 200, NULL, 5, NULL);
    xTaskCreate(uart_task, "UART_Task", 200, NULL, 5, NULL);

    vTaskStartScheduler();

    while(1)
    {
    }
}
```

Above code runs two tasks, led_task and uart_task using FreeRTOS scheduler.

xTaskCreate

```c
int main(void)
{
    CyGlobalIntEnable;

    UART_Start();
    UART_ClearTxBuffer();

    FreeRTOS_Start();

    xTaskCreate(led_task, "LED_Task", 200, NULL, 5, NULL);
    xTaskCreate(uart_task, "UART_Task", 200, NULL, 5, NULL);

    vTaskStartScheduler();

    while(1)
    {
    }
}
```

xTaskCreate(led_task, "LED_Task", 200, NULL, 5, NULL);

Argument	Meaning
`led_task`	Function executed by the task
`"LED_Task"`	Name used for debugging
`200`	Stack depth allocated to the task (in stack units, typically words, not bytes)
`NULL`	No parameters passed to the task
`5`	Task priority
`NULL`	Task handle not required by the caller

→ *If used, task handle can be used to manage the tasks later*
→ *Higher the number, higher the priority*

Scheduler

The scheduler uses priorities and time slicing (for tasks of equal priority) to decide which task runs.

Scheduler Cycle

- The RTOS maintains lists of tasks in different states, including Ready, Blocked, and Suspended.
- It selects the **highest-priority Ready task** to execute.
- The CPU runs that task until one of the following occurs:
  The task blocks — e.g., calls vTaskDelay() or waits for a semaphore or queue.
  A higher-priority task becomes Ready — e.g., when an interrupt unblocks a waiting task.
  A time slice expires — if time slicing is enabled and another task of equal priority is Ready.
- Context switch: The RTOS saves the current task's CPU context (registers, program counter, etc.) on its stack and restores the context of the next task from that task's stack.

E.g.,

Task	Priority
Task-H	3
Task-M	2
Task-L1	1
Task-L2	1

Configuraiton
- Pre-emption: Enabled
- Time-slicing: Enabled

1. **At startup:**
   - All tasks are ready.
   - Scheduler checks priorities → runs **Task-H** first (priority 3).
2. **Task-H calls** `vTaskDelay(10)` → becomes *Blocked* for 10 ticks.
   - Scheduler now picks **Task-M** (next highest priority, 2).
3. **Task-M runs** and then also delays (`vTaskDelay(20)`).
   - Both H and M are now blocked → only **Task-L1** and **Task-L2** remain ready.
4. **Task-L1 and Task-L2** (both priority 1) share the CPU using **time slicing**.
   - Each runs for one tick (e.g., 1 ms) alternately:

```
less

Time →  L1 | L2 | L1 | L2 | ...
```

5. **At tick 10:**
   - Task-H becomes ready again → immediately **preempts** Task-L1/L2 (higher priority).
6. **At tick 20:**
   - Task-M becomes ready, but it runs **only when Task-H is blocked again**.

RTOS Keywords

Concept	Description
Task	Independent thread of execution
Scheduler	Decides which Ready task runs next
Running State	Task currently executing on the CPU
Ready State	Task is ready to execute but waiting for CPU time
Blocked State	Task waits for an event, resource, or timeout; the CPU can run other Ready tasks
Suspended State	Task is explicitly suspended and cannot run until resumed
Preemption	A higher-priority Ready task can preempt a lower-priority Running task, when enabled
Time Slicing	Shares CPU time among Ready tasks of equal priority at tick intervals, when enabled
Context Switch	Saving and restoring task CPU context during a switch
Blocking/ Delay	A task waits for an event or a specified period, entering the Blocked state

Mutex

Application

```
task1()
{
    loop forever
    {
        arrADCBytes = read_ADC;
        uart_write(arrADCBytes)
        delay(50 ms);
    }
}

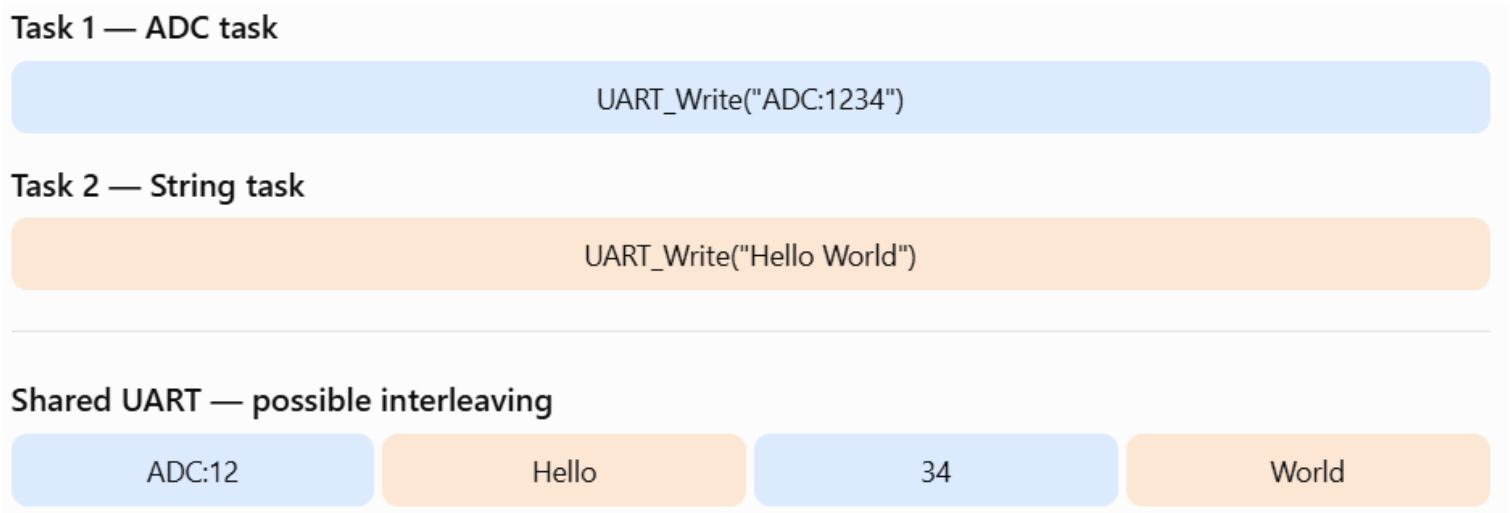
task2()
{
    loop forever
    {
        uart_write("Hello World");
        delay(1 ms);
    }
}
```

- Task-1 and Task-2 log data to the PC using a single UART hardware.
- Task-1 writes ADC output to the UART
- Task-2 writes a string output to the UART

Question – Any Issues ?

Issue

- Both tasks use the same UART peripheral.
- Bytes from the two messages may become interleaved, resulting in corrupted output.



Solution

```
create mutex xUART

task1()
{
    loop forever
    {
        arrADCBytes = read_ADC;

        take xUART
        uart_write(arrADCBytes)
        give xUART

        delay(50 ms);
    }
}
```

```
task2()
{
    loop forever
    {
        take xUART
        uart_write("Hello World");
        give xUART

        delay(1 ms);
    }
}
```

- Both tasks share the same UART peripheral, a critical resource.
- The mutex (xUART) ensures only one task uses UART at a time.
- Prevents interleaved or corrupted UART messages caused by concurrent access.

Semaphores

Application #1

```
Task1()  // SPI Reader Task
{
    loop forever
    {
        arrDataBytes = read_SPI(30 bytes);
        Delay(1.2 ms);
    }
}

Task2()  // UART Sender Task
{
    loop forever
    {
        UART_Write(arrDataBytes);
        Delay(100 ms);
    }
}
```

- Task-1 updates an array with 30bytes from SPI
- Task-2 transmits the array content via UART

Question – Any Issues ?

Issue

```
Task1()  // SPI Reader Task
{
    loop forever
    {
        arrDataBytes = read_SPI(30 bytes);
        Delay(1.2 ms);
    }
}

Task2()  // UART Sender Task
{
    loop forever
    {
        UART_Write(arrDataBytes);
        Delay(100 ms);
    }
}
```

⚠️ The Problem – Data Corruption

- Both tasks share the same buffer: arrDataBytes.
- While Task-1 writes new SPI data, the RTOS may preempt it and run Task-2.
- Task-2 then transmits a partially updated buffer over UART.
- Result: Mixed old and new data → inconsistent or corrupted output.
- Timing-dependent issue — may appear correct sometimes, corrupted at others.

Solution

```
Create binary semaphore: xDataReady;

Task1()  // SPI Reader Task
{
    loop forever
    {
        arrDataBytes = read_SPI(30 bytes);
        Give(xDataReady);
        Delay(1.2 ms);
    }
}

Task2()  // UART Sender Task
{
    loop forever
    {
        Take(xDataReady);
        Copy(arrDataBytes, arrTemp);
        UART_Write(arrTemp);
        Delay(100 ms);
    }
}
```

xDataReady → Binary Semaphore

Give(xDataReady): Signals that data is ready and allows a waiting task to proceed.

Take(xDataReady, WAIT_FOREVER): Blocks the task until the semaphore becomes available.

Semaphore here synchronize Task-1 and Task-2

Requirement: Operation “copy arrDataBytes to arrTemp” should be completed within 1.2ms. **Why ?**

You may also use mutex

Application #2

```
task1()
{
     ...
}

task2()
{
     ...
}

task3()
{
     ...
}
```

Question: Execute Task1, Task2, and Task3 sequentially, in that order, every 10 ms.

How ?

Solution

```
Create binary semaphores: xTask1, xTask2, xTask3

MasterTask()
{
    loop:
        Give(xTask1)
        Delay(10 ms)
}

Task1()
{
    loop:
        Take(xTask1)
        ...
        Give(xTask2)
}

Task2()
{
    loop:
        Take(xTask2)
        ...
        Give(xTask3)
}

Task3()
{
    loop:
        Take(xTask3)
        ...
}
```

Question: Run tasks in the following order, task1 → task2 → task3 → task1 …

Application #3

Question: Execute task1 when switch-1 is pressed and task2 when switch-2 is pressed. **How ?**

```
Create binary semaphore: xTask1
Create binary semaphore: xTask2

Switch1_ISR()
{
    GiveFromISR(xTask1);
}

Switch2_ISR()
{
    GiveFromISR(xTask2);
}
```

```
Task1()
{
    loop forever
    {
        Take(xTask1);
        ...
    }
}

Task2()
{
    loop forever
    {
        Take(xTask2);
        ...
    }
}
```

E.g., ISR Integration

Q: Toggle an LED when a switch is pressed

```
CY_ISR (switch_isr_code)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // Give semaphore from ISR
    xSemaphoreGiveFromISR(xButtonSemaphore, &xHigherPriorityTaskWoken);

    // Request context switch if needed
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);

    Pin_SW_ClearInterrupt();
}

void led_task()
{
    while(1)
    {
        /* Block indefinitely until semaphore is given */
        if (xSemaphoreTake(xButtonSemaphore, portMAX_DELAY) == pdTRUE)
        {
            uint8 led_state = !led_Read();
            led_Write(led_state);
        }

    }
}
```

```
int main(void)
{
    CyGlobalIntEnable;

    isr_1_StartEx(switch_isr_code);

    FreeRTOS_Start();

    xButtonSemaphore = xSemaphoreCreateBinary();

    xTaskCreate(led_task, "LED_Task", 200, NULL, 5, NULL);

    vTaskStartScheduler();



    while(1)
    {
    }
}
```

Mutex vs. Semaphore

Core Differences

Feature	Mutex (Mutual Exclusion)	Semaphore
Purpose	Protect a *shared resource* (e.g., UART, I2C, buffer)	Signal or synchronize between *tasks or ISRs*
Ownership	Has ownership — **the task that takes it must release it**	No ownership — **any task can give or take**
Typical Usage	Prevent simultaneous access to critical section	Task synchronization or event signaling
Priority Inheritance	**Yes. Prevents priority inversion**	No

Priority Inversion - A high-priority task is forced to wait because a low-priority task holds a shared resource (e.g., a semaphore), while a medium-priority task keeps running and prevents the low-priority one from finishing and releasing it.

Priority Inheritance

Priority Inheritance is a mechanism in an RTOS where a low-priority task temporarily gets a higher priority because it holds a mutex needed by a high-priority task.

A low-priority task freezes a high-priority task → Priority Inversion
E.g., when using a Semaphore

- **Task-L (Low priority)** acquires the semaphore to use the UART.
- **Task-H (High priority)** requests the same semaphore and becomes Blocked.
- **Task-M (Medium priority)** becomes Ready and preempts Task-L.
- Task-L cannot finish its UART operation and release the semaphore while Task-M keeps running.
- Task-H remains blocked — a medium-priority task indirectly delays a high-priority task.

Solution: Priority Inheritance
E.g., when using a mutex

1. Task-L takes the mutex.
2. Task-H tries to take the same mutex → blocked.
3. **RTOS temporarily raises Task-L's priority** to match Task-H's priority.
4. Task-L runs immediately, finishes UART operation quickly, and releases the mutex.
5. Task-H resumes — no more inversion!

Queues

Application

```
Create queue: adcQueue (32 items)

ADCTask()  // Producer
{
    loop forever
    {
        sample = Read_ADC()
        xQueueSend (adcQueue, sample)
        Delay(100 ms)
    }
}

UARTTask()  // Consumer
{
    loop forever
    {
        sample = xQueueReceive(adcQueue)
        UART_Write(sample)
    }
}
```

- **Queue**: Transfers data safely between tasks on a FIFO (First In, First Out) basis, without directly sharing a common buffer.
- **xQueueSend():** Adds data to the queue if space is available
- **xQueueReceive()**: Retrieves and removes data from the queue if data is available

E.g., FreeRTOS Code

```c
extern void FreeRTOS_Start();

static QueueHandle_t adcQueue;

typedef struct {
    uint16 index;     // sample index
    uint16 sample;    // 12-bit ADC sample (dummy)
} adc_msg_t;


void adc_task()
{
    adc_msg_t msg_adc;
    uint16 counter=0;

    while(1)
    {
        msg_adc.index = counter; counter++;
        msg_adc.sample = (uint16_t)(rand() & 0x0FFF);


        xQueueSend(adcQueue, &msg_adc, portMAX_DELAY);

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

```c
void uart_task()
{
    while(1)
    {
        adc_msg_t msg_rcv;
        char str[100];

        if (xQueueReceive(adcQueue, &msg_rcv, portMAX_DELAY) == pdTRUE)
        {
            sprintf(str, "ADC %d %d\r\n", msg_rcv.index, msg_rcv.sample);
            UART_PutString(str);
        }

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

```c
int main(void)
{
    CyGlobalIntEnable;

    UART_Start();
    UART_ClearTxBuffer();

    FreeRTOS_Start();

    /* Create queue to hold up to 32 adc_msg_t items */
    adcQueue = xQueueCreate(32, sizeof(adc_msg_t));

    xTaskCreate(adc_task, "ADC Task", 200, NULL, 5, NULL);
    xTaskCreate(uart_task, "UART Task", 200, NULL, 5, NULL);

    vTaskStartScheduler();

    while(1)
    {
    }
}
```

What Next ?

Learn/ Read

- Hands-on Implementation: Running FreeRTOS on PSoC 5LP and ESP32.
- Task Creation: Understanding xTaskCreate() arguments, including stack size, task priority, and task handles.
- Task Timing: Using vTaskDelay() and configuring the RTOS tick interval.
- Scheduling Configuration: Enabling/disabling preemption and time slicing.
- Semaphores and Mutexes: Binary vs. counting semaphores; queues vs. mutexes; using mutexes for task synchronization.
- Inter-task communication: Queues vs. shared buffers protected by mutexes for
- Task Management: Suspending, resuming, deleting, and monitoring tasks.